

Σχεδίαση και Χρήση Βάσεων Δεδομένων - Εαρινό Εξάμηνο 2025

Άσκηση 2

Παράδοση Άσκησης: 18 Μαΐου 2025, Ώρα: 23:59

Σας δίνεται μια εφαρμογή τριών επιπέδων. Αποτελείται από τη διεπαφή χρήστη, που είναι web based, τη λογική της εφαρμογής, που είναι σε Python, και τη Βάση Δεδομένων (ΒΔ), που είναι σε MySQL. Σκοπός είναι να λύσετε τα ερωτήματα χρησιμοποιώντας Python και τη βιβλιοθήκη *pygame*, ενώ τα αποτελέσματα της εργασίας θα φαίνονται στον browser με τη χρήση της βιβλιοθήκης *Bottle*. Ένας επιπλέον στόχος της εργασίας είναι η εξοικείωση με τα μοντέλα γλωσσικής επεξεργασίας και η χρήση τους για την παραγωγή εκτελέσιμου κώδικα. Η υλοποίηση της εργασίας θα γίνει με υβριδικό τρόπο, συνδυάζοντας τη συγγραφή κώδικα και την αλληλεπίδρασή σας με το μοντέλο ChatGPT (<https://chat.openai.com/>).

Πληροφορίες για τη Βάση Δεδομένων

Θα πρέπει να γράψετε και να εκτελέσετε μια σειρά από ερωτήματα στην παρακάτω βάση. Η βάση που θα χρησιμοποιήσετε ονομάζεται Airbnb και σχηματίζεται από τις ακόλουθες σχέσεις:

amenity (amenity_id int(11) PK, amenity_name varchar(100))

- amenity_id = το id μιας παροχής (PK)
- amenity_name = το όνομα μιας παροχής

booking (booking_id int(11) PK, guest_id int(11), property_id int(11), check_in date, check_out date)

- booking_id = το id μιας κράτησης (PK)
- guest_id = το id ενός φιλοξενούμενου
- property_id = το id ενός ξενώνα
- check_in = ημερομηνία έναρξης της κράτησης
- check_out = ημερομηνία λήξης της κράτησης

guest (guest_id int(11) PK, first_name varchar(50), last_name varchar(50), email varchar(100))

- guest_id = το id ενός φιλοξενούμενου (PK)
- first_name = το όνομά του
- last_name = το επώνυμό του
- email = ο λογαριασμός ηλεκτρονικού ταχυδρομείου του

host (host_id int(11) PK, first_name varchar(50), last_name varchar(50), phone_number varchar(20))

- host_id = το id ενός ιδιοκτήτη ξενώνα (PK)
- first_name = το όνομά του
- last_name = το επώνυμό του
- phone_number = ο αριθμός τηλεφώνου του

houserule (rule_id int(11) PK, rule_name varchar(100))

- rule_id = το id ενός κανόνα σε ξενώνα (PK)
- rule_name = το όνομα του κανόνα

property (property_id int(11) PK, name varchar(100), location varchar(255), rating decimal(2,1), price decimal(10,2), host_id int(11))

- property_id = το id ενός ξενώνα (PK)
- name = το όνομά του
- location = η πόλη στην οποία βρίσκεται
- rating = η βαθμολογία του ξενώνα
- price = η τιμή του ανά βραδιά
- host_id = το id ενός ιδιοκτήτη ξενώνα

property_has_amenity (property_id int(11) PK, amenity_id int(11) PK)

- property_id, amenity_id = συσχετίζει ένα ξενώνα με μια παροχή (PK)

property_has_rule (property_id int(11) PK, rule_id int(11) PK)

- property_id, rule_id = συσχετίζει ένα ξενώνα με ένα κανόνα (PK)

property_has_type (property_id int(11) PK, type_id int(11) PK)

- property_id, type_id = συσχετίζει ένα ξενώνα με ένα τύπο ξενώνα (PK)

propertytype (type_id int(11) PK, type_name varchar(50))

- type_id = το id ενός τύπου ξενώνα (PK)
- type_name = το όνομα του τύπου ενός ξενώνα

review (review_id int(11) PK, guest_id int(11), property_id int(11), rating decimal(2,1), comment text, created_at timestamp)

- review_id = το id μιας αξιολόγησης (PK)
- guest_id = το id ενός φιλοξενούμενου
- property_id = το id ενός ξενώνα
- rating = ο βαθμός αξιολόγησης
- comment = σχόλια αξιολόγησης
- created_at = ημερομηνία δημιουργίας της αξιολόγησης

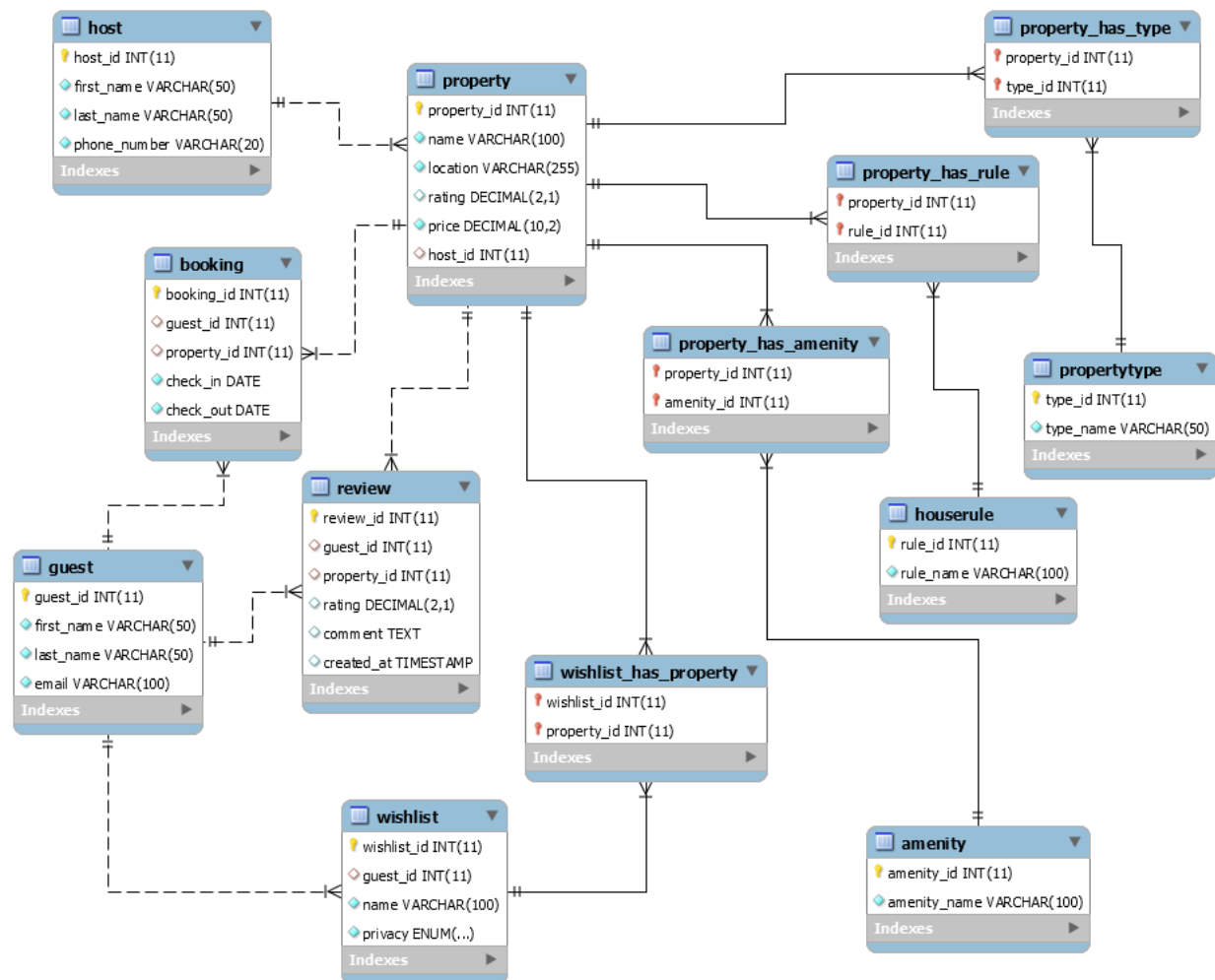
wishlist (wishlist_id int(11) PK, guest_id int(11), name varchar(100), privacy enum('Public','Private'))

- wishlist_id = το id μιας λίστας επιθυμιών ενός φιλοξενούμενου (PK)
- guest_id = το id ενός φιλοξενούμενου
- name = το όνομα της λίστας επιθυμιών
- privacy = χαρακτηρισμός σε δημόσια ή ιδιωτική λίστα

wishlist_has_property (wishlist_id int(11) PK, property_id int(11) PK)

- wishlist_id, property_id = συσχετίζει μια λίστα επιθυμιών με ένα φιλοξενούμενο (PK)

Ακολουθεί το σχήμα της βάσης σε ER διάγραμμα:



Οδηγίες

Για να τρέξετε αρχικά την εργασία σας, θα πρέπει να κάνετε τα εξής:

- Να μελετήσετε τις διαφάνειες του φροντιστηρίου (Εργαστήριο 2) και να εξοικειωθείτε με την MySQL.
- Να μελετήσετε τις διαφάνειες του φροντιστηρίου (Εργαστήριο 3). Εκεί θα βρείτε οδηγίες εγκατάστασης για την Python 3.xx και το Bottle web framework, καθώς και βασικά παραδείγματα εξοικείωσης με web-based εφαρμογή. Για την παρούσα εργασία, θα χρησιμοποιηθεί το Bottle ως web framework.

- Να κάνετε unzip το application.zip.
- Να αλλάξετε ό,τι χρειάζεται στη μεταβλητή db_config στο αρχείο app.py, ώστε να μπορείτε να συνδεθείτε στη δική σας ΒΔ MySQL
- Να τρέξετε το website.py με την Python.
- Τέλος, να ανοίξετε κάποιον browser και να βάλετε τη διεύθυνση “<http://localhost:8080>”

Το παρακάτω είναι η αρχική σελίδα που πρέπει να δείτε:

Check If Property Exists Location: Property type: Update	High Value Host min price: min rating: min avg price host: min avg rating host: Update	Find Common Properties And Guests guestIdA: guestIdB: Search
Count Words For Properties N: M: Search	Top N Hosts N: Search	
Find Matching Properties Guest Id: Search	Recommend Property Guest Id: City: Amenities: max price: min rating: Search	

Καλείστε να αλλάξετε τη λογική της εφαρμογής η οποία βρίσκεται στο app.py έτσι ώστε να εκτελεί τα παρακάτω ζητούμενα. Όλες οι συναρτήσεις μέσα στο αρχείο app.py επιστρέφουν μια λίστα από πλειάδες (tuples) όπου πάντα η πρώτη πλειάδα είναι η κεφαλίδα με τα ονόματα των πεδίων και οι υπόλοιπες είναι τα αποτελέσματα.

Για παράδειγμα: [("Name", "Id"), ("Jim", 7), ("Tom", 13,)]

Περιγραφή των συναρτήσεων:

checkIfPropertyExists: Η συνάρτηση αυτή δέχεται ως όρισμα την τοποθεσία (location_a) και έναν τύπο καταλύματος (property_type_a). Ελέγξτε αν υπάρχει ιδιοκτησία στην τοποθεσία **location_a** που είναι τύπου **property_type_a**. Το ερώτημα θα πρέπει να επιστρέφει ως απάντηση τιμή “yes” ή “no”. Απαγορεύεται η χρήση Flow Control Operators (δηλαδή if, case, κ.λπ).

selectTopNhosts: Η συνάρτηση αυτή δέχεται ως όρισμα έναν ακέραιο **N**. Για κάθε **τύπο καταλύματος**, βρίσκει τους **N ιδιοκτήτες (hosts)** που έχουν καταχωρήσει τα περισσότερα καταλύματα αυτού του τύπου. Επιστρέφει τα εξής:

- Το **όνομα του τύπου καταλύματος**.
- Το **ID** κάθε ιδιοκτήτη.
- Τον **αριθμό των καταλυμάτων** αυτού του τύπου που έχει καταχωρήσει ο κάθε ιδιοκτήτης.

findMatchingProperties: Ένας επισκέπτης θέλει να βρει καταλύματα που πληρούν συγκεκριμένες προϋποθέσεις: - είσοδος `guest_id`

- Το κατάλυμα πρέπει να έχει τουλάχιστον μία παροχή (**amenity**) που να υπάρχει και σε κάποιο άλλο κατάλυμα στη λίστα επιθυμιών του.
- Το κατάλυμα πρέπει να έχει τουλάχιστον έναν κανόνα διαμονής (**house rule**) κοινό με κάποιο κατάλυμα στο οποίο έχει ήδη κάνει κράτηση.
- Ο οικοδεσπότης αυτού του καταλύματος **δεν πρέπει να είναι ο ίδιος** με οποιονδήποτε οικοδεσπότη καταλυμάτων στα οποία έχει κάνει κράτηση στο παρελθόν.

Τα αποτελέσματα πρέπει να περιλαμβάνουν:

- Το **property_id** και το όνομα κάθε καταλύματος που ικανοποιεί τα κριτήρια.
- Τις παροχές (**amenities**).
- Τους κανόνες διαμονής (**house rules**).

countWordsForProperties: Η συνάρτηση αυτή παίρνει ως όρισμα δύο αριθμητικές τιμές **N** και **M**. Επιστρέφει καταλύματα που πληρούν συγκεκριμένες προϋποθέσεις:

- Το ακίνητο πρέπει να έχει κρατηθεί τουλάχιστον από **N διαφορετικούς επισκέπτες** στο παρελθόν.
- Τουλάχιστον ένας από αυτούς τους επισκέπτες πρέπει να έχει αφήσει κριτική (**review**).
- Το ακίνητο πρέπει να έχει τουλάχιστον δύο παροχές (**amenities**).
- Το ακίνητο **δεν πρέπει να βρίσκεται** σε καμία λίστα επιθυμιών (**wishlist**).

Τα αποτελέσματα πρέπει να περιλαμβάνουν:

- Ένα μήνυμα στην αρχή (ή στο τέλος) που να εμφανίζει τον συνολικό αριθμό των καταχωρήσεων ακινήτων που πληρούν τις συνθήκες.
- Το `property_id`, το όνομα (`name`) και την τοποθεσία (`location`) των αντίστοιχων ακινήτων.
- Τον συνολικό αριθμό των **μοναδικών** επισκεπτών που το έχουν κλείσει.
- Τις παροχές (**amenities**) του ακινήτου.
- Τέλος, να συλλεχθούν όλες οι κριτικές για το ακίνητο και να δημιουργηθεί μια λίστα με τις πιο συχνά χρησιμοποιούμενες λέξεις (**M**) σε αυτές τις κριτικές αφού πρώτα αφαιρέσετε τα `stop words` (πχ τις λέξεις `and`, `or`, `not`, `to`, `at`, κτλ).

findCommonPropertiesAndGuests: Η συνάρτηση αυτή παίρνει ως όρισμα ένα ζευγάρι (`a,b`) από κωδικούς επισκεπτών (`guest_id a`, `guest_id b`). Βρίσκει τα **κοινά** καταλύματα που έχουν μείνει δύο διαφορετικοί επισκέπτες (`c`, `d`) με την προϋπόθεση ότι ο `c` έχει μείνει σε ένα ακίνητο που είχε μείνει και ο `a`, και ο `d` έχει μείνει σε ένα ακίνητο που είχε μείνει ο `b`.

Επιστρέφει τα εξής:

- Τον τίτλο του κάθε ακινήτου για το ζευγάρι (`c`, `d`).
- Τον κωδικό του `c`.
- Τον κωδικό του `d`.
- Τον κωδικό του `a`.
- Τον κωδικό του `b`.

highValueHost: Στο ερώτημα αυτό θα δημιουργήσετε μια συνάρτηση που επιστρέφει τις δημοφιλέστερες παροχές μεταξύ των επισκεπτών που προτιμούν καταλύματα “υψηλής αξίας”. Η συνάρτηση δέχεται ως ορίσματα:

- `min_price_booking` (double): Η ελάχιστη τιμή που πρέπει να έχει κλείσει ένας επισκέπτης κατάλυμα για να θεωρηθεί επισκέπτης "υψηλής αξίας".
- `min_rating_review` (double): Η ελάχιστη βαθμολογία που πρέπει να έχει δώσει ένας επισκέπτης σε μια κριτική για να θεωρηθεί επισκέπτης "υψηλής αξίας".
- `min_avg_price_host` (double): Η ελάχιστη μέση τιμή καταλύματος που πρέπει να έχει ένας οικοδεσπότης για να θεωρηθεί οικοδεσπότης "υψηλής αξίας".
- `min_avg_rating_host` (double): Η ελάχιστη μέση βαθμολογία καταλύματος που πρέπει να έχει ένας οικοδεσπότης για να θεωρηθεί οικοδεσπότης "υψηλής αξίας".

Σημείωση : Ο όρος "μέση" τιμή αναφέρεται στον μέσο όρο των τιμών όλων των καταλυμάτων του οικοδεσπότη.

Βρείτε τις παροχές (amenities) που σχετίζονται με τα καταλύματα των οικοδεσποτών που έχουν μέση τιμή καταλύματος μεγαλύτερη ή ίση με `min_avg_price_host` και μέση βαθμολογία καταλύματος μεγαλύτερη ή ίση με `min_avg_rating_host`, και ταυτόχρονα έχουν κλειστεί από επισκέπτες που έχουν κλείσει καταλύματα με τιμή μεγαλύτερη ή ίση με `min_price_booking` και έχουν γράψει κριτικές με βαθμολογίες μεγαλύτερες ή ίσες με `min_rating_review`. Επιστρέψτε μια λίστα από πλειάδες, όπου κάθε πλειάδα περιέχει το **amenity_name** (όνομα παροχής) και τη συχνότητα αυτής της παροχής, ταξινομημένες κατά **φθίνουσα** σειρά.

recommendProperty: Ένας επισκέπτης σχεδιάζει ένα ταξίδι και θέλει να βρει το "καλύτερο" κατάλυμα για κράτηση, βάσει διαφόρων κριτηρίων:

- **Επιθυμητές Παροχές:** Ο επισκέπτης παρέχει μια λίστα με τις επιθυμητές παροχές, όπου κάθε παροχή έχει ένα βάρος που υποδεικνύει τη σημασία της.
- **Πόλη Καταλύματος:** Ο επισκέπτης θέλει να βρει καταλύματα σε μια συγκεκριμένη πόλη (location).
- **Βαθμολογία Καταλύματος:** Ο επισκέπτης προτιμά καταλύματα με υψηλότερες βαθμολογίες.
- **Τιμή Καταλύματος:** Ο επισκέπτης θέλει να μείνει εντός ενός καθορισμένου εύρους τιμών.

Γράψτε μια συνάρτηση που δέχεται τις ακόλουθες εισόδους:

- `guest_id`: Το ID του επισκέπτη.
- `desired_location`: Το όνομα της πόλης όπου ο επισκέπτης θέλει να μείνει.
- `desired_amenities`: Ένα λεξικό όπου τα κλειδιά είναι τα ονόματα των παροχών και οι τιμές είναι τα βάρη (π.χ., {"WiFi": 3, "Balcony": 2, "Elevator": 4}).
- `max_price`: Η μέγιστη τιμή που ο επισκέπτης είναι διατεθειμένος να πληρώσει.
- `min_rating`: Η ελάχιστη βαθμολογία που επιθυμεί ο επισκέπτης.

Η συνάρτηση πρέπει να **προτείνει το βέλτιστο κατάλυμα**: Επιστρέφει το **property_id** και το **property_name** του καταλύματος με την υψηλότερη συνδυασμένη βαθμολογία και στη συνέχεια **δημιουργεί μία νέα wishlist και την εισάγει στη ΒΔ σας**, για τον επισκέπτη που δόθηκε στην

αρχή. Η wishlist μπορεί να είναι είτε Private, είτε Public, ενώ πρέπει να φροντίσετε ώστε η εισαγωγή στη ΒΔ να ολοκληρωθεί σωστά και να ενημερωθούν όλοι οι σχετικοί πίνακες. Σε περίπτωση επιτυχίας, επιστρέφει “ok”. Αν δεν υπάρχει βέλτιστο κατάλυμα, ή αν δεν ολοκληρωθεί το INSERT να επιστρέφει ένα μήνυμα λάθους.

Για να βρεθεί το βέλτιστο κατάλυμα, πρέπει η συνάρτηση να βρίσκει καταλύματα στην καθορισμένη πόλη, εντός του καθορισμένου εύρους τιμών και βαθμολογιών. Δηλαδή, τα αποτελέσματα της συνάρτησης αυτής μπορεί να είναι και περισσότερα από ένα καταλύματα. Οι βαθμολογίες υπολογίζονται ως εξής:

- **Υπολογίζει βαθμολογίες για τις παροχές:** Υπολογίζει μια βαθμολογία για κάθε κατάλυμα με βάση το σταθμισμένο άθροισμα (weighted sum) των επιθυμητών παροχών.
- **Υπολογίζει μια συνδυασμένη βαθμολογία:** Συνδυάζει τη βαθμολογία παροχών (amenities) και τη βαθμολογία του καταλύματος (rating) σε μια ενιαία βαθμολογία. Θα χρειαστεί να αποφασίσετε για μια κατάλληλη συνάρτηση βαθμολόγησης που εξισορροπεί αυτούς τους παράγοντες.

Επιλογή Συνάρτησης Βαθμολόγησης για τις παροχές (επιλέξτε μία):

Επιλέξτε μία από τις ακόλουθες επιλογές για τη συνάρτηση βαθμολόγησης για τις παροχές:

- *Σταθμισμένο Άθροισμα (Γραμμικός Συνδυασμός):*
 - Κάθε παράγοντας πολλαπλασιάζεται με ένα βάρος που αντιπροσωπεύει τη σημασία του και τα αποτελέσματα αθροίζονται.
 - Τύπος: $\text{Βαθμολογία} = (w1 * \text{factor}_1) + (w2 * \text{factor}_2) + \dots + (wn * \text{factor}_n)$
 - w_i : Βάρος για τον παράγοντα i
 - factor_i : Τιμή του παράγοντα i
- *Κανονικοποιημένο Σταθμισμένο Άθροισμα:*
 - Οι παράγοντες κανονικοποιούνται (κλιμακώνονται σε ένα κοινό εύρος, π.χ., 0 έως 1 πριν συνδυαστούν).
 - Τύπος: $\text{Βαθμολογία} = (w1 * \text{normalized_factor1}) + (w2 * \text{normalized_factor2}) + \dots + (wn * \text{normalized_factorN})$
 - Κανονικοποίηση:
 - Min-max scaling: $\text{normalized_factor} = (\text{factor} - \text{min_factor}) / (\text{max_factor} - \text{min_factor})$
 - Κανονικοποίηση Z-score: $\text{normalized_factor} = (\text{factor} - \text{mean_factor}) / \text{std_factor}$

Επιλογή συνάρτησης για την συνδυασμένη βαθμολογία (επιλέξτε μία):

- Απλός γραμμικός συνδυασμός με βάρη
 - $\text{Βαθμολογία} = (\text{rating} * w1) + (\text{amenity_score} * w2)$, όπου τα $w1, w2$ θα είναι δική σας επιλογή
- Εκθετική συνάρτηση:

- $\text{Βαθμολογία} = (\text{rating} \wedge \text{w_rating}) * (\text{amenity_score} \wedge \text{w_amenities}) / (\text{price} \wedge \text{w_price})$. Το w_price θα είναι δικής σας επιλογής και μπορεί να είναι είτε θετικό, το οποίο πρακτικά σημαίνει ότι θα προκρίνονται τα πιο φθηνά καταλύματα, είτε αρνητικό, δηλαδή θα προκρίνονται τα πιο ακριβά).

Γενικά ζητήματα που πρέπει να προσέξετε

- Είναι σημαντικό να μελετήσετε/τρέξετε τα παραδείγματα του φροντιστηρίου πριν ξεκινήσετε την υλοποίηση της εργασίας.
 - Δεν επιτρέπεται καμία εντολή που δεν έχετε διδαχθεί στο μάθημα. Ενδεικτικά παραδείγματα που δεν επιτρέπονται είναι τα παρακάτω:
Τα SELECT μέσα στο SELECT και το FROM clause δεν επιτρέπονται.
Το JOIN δεν επιτρέπεται καθόλου ως εντολή (δηλαδή δεν επιτρέπονται LEFT JOIN, CROSS JOIN, INNER JOIN κτλ...). Δεν επιτρέπεται η χρήση της LIMIT.
 - Μπορείτε να χρησιμοποιήσετε περισσότερα από ένα SQL ερωτήματα. Τα ερωτήματα μπορείτε φυσικά να τα δοκιμάζεται και στο Workbench όμως στο τελικό παραδοτέο πρέπει να τρέχουν μόνο μέσω των αρχείων που σας δίνουμε σε pytho, διαφορετικά δεν βαθμολογούνται.
 - Οτιδήποτε άλλο αφορά την pytho επιτρέπεται (εκτός από την βιβλιοθήκη pandas).
 - Στο αρχείο app.py έχετε έναν σκελετό για την εργασία και θα χρειαστεί να τροποποιήσετε τις συναρτήσεις που δίνονται ώστε να υλοποιηθούν τα ζητούμενα. Αντιθέτως τα υπόλοιπα αρχεία στον φάκελο application.zip δεν χρειάζεται να τροποποιηθούν.
 - Θα παραδώσετε την εφαρμογή σας, δηλαδή το αρχείο **app.py**, και ένα **README** σε ένα αρχείο της μορφής sdixxxx_app.zip (τα υπόλοιπα αρχεία που περιέχονται στον φάκελο application.zip, δεν χρειάζεται να συμπεριληφθούν στο παραδοτέο της εργασίας). Το αρχείο README θα περιέχει
 - Για κάθε συνάρτηση, να μας δώσετε το αντίστοιχο σύνδεσμο που περιέχει την συνομιλία σας με το ChatGPT από την οποία προέκυψε η συγκεκριμένη συνάρτηση ή δομή. Ο σύνδεσμος αυτός προκύπτει από την κεντρική σελίδα του ChatGPT χρησιμοποιώντας το κουμπί στο δεξί μέρος μιας συνομιλίας:
-
- Προσοχή! ΜΗΝ διαγράψετε την συνεδρία γιατί θα χαθεί και η συγκεκριμένη συνομιλία με αποτέλεσμα να χάσετε μονάδες από την βαθμολογία σας.
 - Για κάθε συνάρτηση θα αναφέρετε συνοπτικά τις αλλαγές που κάνατε στον παραγόμενο από το ChatGPT κώδικα.
- Η εργασία πρέπει να εμφανίζει τα αποτελέσματα στον browser, στη διεύθυνση "http://localhost:8080".

Καλή Επιτυχία!